

Instituto Federal Catarinense

Campus Rio do Sul

Curso de Ciência da Computação

Disciplina: Estruturas de Dados I

Professor: Daniel Gomes Soares

Trabalho 2

Análise de Métodos de Ordenação

Estudante: Carlos Gabriel Fontanive

Data de entrega: 06 de julho de 2026

Rio do Sul – SC
2026

Sumário

1 Introdução **2**

2 Descrição dos algoritmos **2**

2.1 Bubble Sort 2

2.2 Insertion Sort 3

2.3 Selection Sort 3

2.4 Merge Sort 3

2.5 Quick Sort 3

2.6 Heap Sort 4

2.7 Shell Sort 5

3 Metodologia dos testes **6**

3.1 Ambiente de execução 6

3.2 Critérios de teste 6

3.3 Tempo total de execução 7

4 Gráficos e tabelas de desempenho **7**

4.1 Vetor ordenado 8

4.2 Vetor invertido 9

4.3 Vetor aleatório 10

4.4 Gráficos de tempo de execução 11

5 Análise dos resultados **12**

6 Código-fonte **13**

6.1 Trechos relevantes do código 14

7 Conclusão **16**

1 Introdução

Os algoritmos de ordenação são fundamentais na computação, pois aparecem em diversas situações práticas, como organização de dados, consultas em bancos de dados, sistemas de busca, processamento de arquivos, análise de grandes volumes de informações e otimização de estruturas de dados. A ordenação permite que outras operações sejam realizadas de forma mais eficiente, como buscas, comparações, agrupamentos e geração de relatórios.

O objetivo deste trabalho é implementar, analisar e comparar diferentes métodos de ordenação utilizando a linguagem Java. Foram considerados os algoritmos Bubble Sort, Insertion Sort, Selection Sort, Merge Sort e Quick Sort, além de dois algoritmos adicionais pesquisados: Heap Sort e Shell Sort.

A análise de desempenho foi realizada por meio da contagem de comparações, movimentações e tempo de execução. Para tornar os testes mais completos, os algoritmos foram executados em diferentes tamanhos de entrada e em três tipos de vetores: já ordenado, em ordem inversa e com valores aleatórios. Dessa forma, foi possível observar como cada algoritmo se comporta em situações distintas e verificar se os resultados práticos se aproximam da complexidade teórica esperada.

2 Descrição dos algoritmos

Nesta seção são descritos os algoritmos de ordenação implementados no trabalho. Os cinco primeiros fazem parte do conjunto principal solicitado, enquanto Heap Sort e Shell Sort foram escolhidos como algoritmos adicionais pesquisados.

2.1 Bubble Sort

O Bubble Sort é um algoritmo simples de ordenação baseado na comparação entre elementos vizinhos. A cada passagem pelo vetor, elementos adjacentes são comparados e, caso estejam fora de ordem, são trocados de posição. Esse processo faz com que os maiores elementos sejam levados gradualmente para o final do vetor.

Apesar de ser fácil de compreender e implementar, o Bubble Sort apresenta baixo desempenho em vetores grandes. Sua complexidade média e no pior caso é $O(n^2)$, pois o algoritmo realiza muitas comparações entre elementos vizinhos. No melhor caso, quando o vetor já está ordenado e a implementação utiliza uma verificação de troca, o algoritmo pode encerrar antes, apresentando comportamento próximo de $O(n)$.

2.2 Insertion Sort

O Insertion Sort funciona construindo gradualmente uma parte ordenada do vetor. A cada iteração, um elemento é escolhido e inserido na posição correta em relação aos elementos anteriores. Esse funcionamento é semelhante à organização manual de cartas em uma mão.

Esse algoritmo costuma ter bom desempenho em vetores pequenos ou quase ordenados, pois nesses casos poucas movimentações são necessárias. Entretanto, em vetores invertidos ou muito desordenados, o número de deslocamentos aumenta bastante. Sua complexidade no pior caso é $O(n^2)$, mas no melhor caso pode se aproximar de $O(n)$.

2.3 Selection Sort

O Selection Sort ordena o vetor selecionando, a cada passo, o menor elemento da parte ainda não ordenada e colocando-o na posição correta. Primeiro encontra-se o menor valor do vetor, troca-se com a primeira posição, depois busca-se o menor valor restante e assim sucessivamente.

Uma característica importante do Selection Sort é que ele realiza poucas trocas em comparação com Bubble Sort, pois normalmente faz no máximo uma troca por posição. No entanto, ele continua realizando muitas comparações, mesmo quando o vetor já está ordenado. Por isso, sua complexidade é $O(n^2)$ no melhor, médio e pior caso.

2.4 Merge Sort

O Merge Sort é um algoritmo eficiente baseado na estratégia de divisão e conquista. Ele divide o vetor em duas partes, ordena cada metade recursivamente e depois combina os resultados por meio de um processo de intercalação.

A principal vantagem do Merge Sort é sua complexidade $O(n \log n)$ em todos os casos: melhor, médio e pior. Isso garante um desempenho estável mesmo em vetores grandes. Por outro lado, uma limitação do Merge Sort é o uso de memória auxiliar, pois geralmente é necessário criar um vetor adicional para realizar a intercalação dos elementos.

2.5 Quick Sort

O Quick Sort também utiliza a estratégia de divisão e conquista. Ele escolhe um elemento chamado pivô e reorganiza o vetor de modo que os elementos menores fiquem de um lado e os maiores fiquem do outro. Em seguida, o mesmo processo é aplicado recursivamente às partições formadas.

Na prática, o Quick Sort costuma ser muito rápido, principalmente em vetores aleatórios, apresentando complexidade média $O(n \log n)$. Porém, dependendo da escolha do pivô e da configuração dos dados, pode atingir o pior caso $O(n^2)$. Neste trabalho, o pivô foi

escolhido a partir da posição central do vetor, reduzindo a chance de partições muito desbalanceadas em vetores já ordenados ou invertidos.

2.6 Heap Sort

O Heap Sort foi escolhido como um dos algoritmos adicionais por ser um método clássico de ordenação baseado na estrutura de dados heap. Essa estrutura também é importante em outros contextos da computação, como filas de prioridade e problemas que exigem acesso eficiente ao maior ou menor elemento de um conjunto. Por esse motivo, o Heap Sort é frequentemente apresentado em estudos sobre algoritmos e estruturas de dados [1].

A ideia principal do Heap Sort é transformar o vetor em um heap máximo, isto é, uma estrutura em que o maior elemento fica na raiz. Depois disso, o maior valor é trocado com o último elemento da parte ainda não ordenada. Em seguida, o tamanho ativo do heap é reduzido e a estrutura é reorganizada. Esse processo é repetido até que todo o vetor esteja ordenado.

As etapas principais do Heap Sort são:

1. Construir um heap máximo a partir do vetor original;
2. Trocar o maior elemento, localizado na raiz, com o último elemento da parte não ordenada;
3. Reduzir o tamanho considerado do heap;
4. Reorganizar o heap para manter sua propriedade;
5. Repetir o processo até que o vetor esteja ordenado.

Como exemplo simples, considere o vetor:

$$[4, 1, 3, 2]$$

Após construir o heap máximo, o maior elemento fica na raiz. Em seguida, ele é levado para o final do vetor, e o heap é reorganizado para encontrar o próximo maior elemento. Ao repetir esse processo, o vetor final fica:

$$[1, 2, 3, 4]$$

Entre suas vantagens, destacam-se a complexidade $O(n \log n)$ no melhor, médio e pior caso e o fato de ser um algoritmo *in-place*, ou seja, não necessita de um vetor auxiliar grande. Como limitação, pode realizar mais movimentações que outros algoritmos eficientes e, na prática, pode ser menos rápido que o Quick Sort em algumas entradas.

O Heap Sort é adequado para situações em que se deseja garantir bom desempenho mesmo no pior caso e evitar uso elevado de memória auxiliar.

2.7 Shell Sort

O Shell Sort foi escolhido como segundo algoritmo adicional por ser uma evolução direta do Insertion Sort. Ele foi proposto por Donald Shell e ficou conhecido por apresentar uma ideia simples e eficiente: em vez de comparar apenas elementos vizinhos, o algoritmo começa comparando elementos mais distantes entre si. Essa abordagem reduz deslocamentos longos e pode melhorar o desempenho prático em relação aos métodos quadráticos simples [2].

Enquanto o Insertion Sort trabalha com intervalo 1 desde o início, o Shell Sort utiliza intervalos maiores, chamados de *gaps*. Esses intervalos vão diminuindo até chegar ao valor 1. Quando isso ocorre, o vetor já está parcialmente organizado, tornando a etapa final mais eficiente.

As etapas principais do Shell Sort são:

1. Definir um intervalo inicial, geralmente metade do tamanho do vetor;
2. Comparar e movimentar elementos separados por esse intervalo;
3. Reduzir o intervalo progressivamente;
4. Repetir o processo até o intervalo chegar a 1;
5. Finalizar com uma ordenação semelhante ao Insertion Sort.

Como exemplo simples, considere o vetor:

$$[8, 5, 3, 1, 6, 2]$$

Inicialmente, o algoritmo compara elementos mais distantes, aproximando-os de suas posições corretas. Depois, com intervalos menores, o vetor vai sendo refinado até ficar completamente ordenado:

$$[1, 2, 3, 5, 6, 8]$$

A principal vantagem do Shell Sort é apresentar desempenho prático melhor que Bubble Sort, Selection Sort e Insertion Sort, especialmente em vetores maiores ou parcialmente desordenados. Sua limitação é que sua complexidade depende da sequência de intervalos utilizada, tornando sua análise teórica menos direta que a do Merge Sort ou Heap Sort.

O Shell Sort é adequado quando se busca uma solução simples, com baixo uso de memória e desempenho prático superior aos métodos quadráticos básicos.

3 Metodologia dos testes

Os algoritmos foram implementados na linguagem Java, de forma separada e organizada, utilizando uma interface comum chamada `Sorter`. Cada algoritmo recebeu um vetor de inteiros e um objeto de métricas responsável por armazenar a quantidade de comparações, movimentações e tempo de execução.

Para garantir uma comparação justa, foi gerado um vetor original para cada cenário de teste. Antes da execução de cada algoritmo, uma cópia desse vetor era criada. Dessa forma, todos os algoritmos ordenaram exatamente os mesmos dados, evitando que um algoritmo recebesse um vetor diferente dos demais.

3.1 Ambiente de execução

Os testes foram realizados no ambiente IntelliJ IDEA, com memória máxima disponibilizada para a JVM de `-Xmx12g`. A configuração utilizada foi a seguinte:

Tabela 1: Ambiente de execução dos testes

Item	**Especificação**
Processador	Intel Core i5 de 13ª geração
Memória RAM	24 GB
Armazenamento	SSD NVMe de 512 GB
Velocidade do SSD	Leitura seq.: 4.762,66 MB/s; gravação seq.: 4.347,98 MB/s
Ambiente	IntelliJ IDEA
Memória da JVM	`-Xmx12g`

3.2 Critérios de teste

Os tamanhos de vetores analisados foram:

- 100.000 elementos;
- 1.000.000 de elementos.

Para cada tamanho, foram utilizados três cenários:

- Vetor já ordenado;
- Vetor em ordem inversa;
- Vetor com valores aleatórios.

O tempo de execução foi medido utilizando o método `System.nanoTime()`, considerando apenas o intervalo de execução do algoritmo de ordenação. Portanto, o tempo de criação, preenchimento e cópia dos vetores não foi incluído na medição.

Foram registradas as seguintes métricas:

- Quantidade de comparações entre elementos;
- Quantidade de trocas ou movimentações;
- Tempo de execução em milissegundos.

Cada teste foi executado três vezes, e os resultados foram organizados em arquivos CSV. O arquivo `resultados.csv` contém os resultados individuais de cada execução, enquanto o arquivo `medias.csv` contém as médias agrupadas por algoritmo, tamanho do vetor e tipo de entrada.

Durante os testes, foi verificado se o vetor resultante estava corretamente ordenado ao final da execução de cada algoritmo. Essa verificação foi utilizada apenas como controle de correção e não foi incluída no tempo de execução medido.

3.3 Tempo total de execução

Além dos tempos individuais de cada algoritmo, também foi calculado o tempo total acumulado de ordenação. Esse valor corresponde à soma dos tempos registrados no arquivo `resultados.csv`, considerando apenas o período de execução dos algoritmos de ordenação.

Tabela 2: Tempo total acumulado dos testes de ordenação

Métrica	**Valor**
Quantidade total de execuções	126 execuções
Tempo total em milissegundos	12.781.204,239 ms
Tempo total em segundos	12.781,204 s
Tempo total em minutos	213,02 min
Tempo aproximado em horas	3h 33min 01s

O tempo total informado considera apenas o tempo de ordenação medido com `System.nanoTime()`, sem incluir a criação, preenchimento ou cópia dos vetores.

4 Gráficos e tabelas de desempenho

Nesta seção são apresentados os resultados médios obtidos a partir das execuções dos algoritmos. Os dados foram organizados por cenário, considerando comparações, movimentações e tempo médio de execução.

4.1 Vetor ordenado

Tabela 3: Desempenho médio dos algoritmos com vetor ordenado

Algoritmo	**n**	**Comparações**	**Movimentações**	**Tempo médio (ms)**
Bubble Sort	100.000	99.999	0	1,055
Bubble Sort	1.000.000	999.999	0	0,186
Insertion Sort	100.000	99.999	200.000	1,863
Insertion Sort	1.000.000	999.999	2.000.000	1,118
Selection Sort	100.000	4.999.950.000	0	1.115,347
Selection Sort	1.000.000	499.999.500.000	0	448.606,411
Merge Sort	100.000	0	1.668.928	2,018
Merge Sort	1.000.000	0	19.951.424	8,021
Quick Sort	100.000	1.600.016	0	3,535
Quick Sort	1.000.000	19.000.019	0	10,078
Heap Sort	100.000	249.995	599.991	1,077
Heap Sort	1.000.000	2.499.995	5.999.991	2,122
Shell Sort	100.000	1.500.006	3.000.012	2,950
Shell Sort	1.000.000	18.000.007	36.000.014	14,679

4.2 Vetor invertido

Tabela 4: Desempenho médio dos algoritmos com vetor invertido

Algoritmo	**n**	**Comparações**	**Movimentações**	**Tempo médio (ms)**
Bubble Sort	100.000	4.999.950.000	14.999.850.000	5.623,400
Bubble Sort	1.000.000	499.999.500.000	1.499.998.500.000	556.441,257
Insertion Sort	100.000	9.999.900.000	200.000	3.292,727
Insertion Sort	1.000.000	999.999.000.000	2.000.000	366.232,399
Selection Sort	100.000	4.999.950.000	150.000	4.951,261
Selection Sort	1.000.000	499.999.500.000	1.500.000	772.371,071
Merge Sort	100.000	0	1.668.928	0,672
Merge Sort	1.000.000	0	19.951.424	7,057
Quick Sort	100.000	1.600.030	150.000	2,409
Quick Sort	1.000.000	19.000.036	1.500.000	11,020
Heap Sort	100.000	149.998	300.000	0,136
Heap Sort	1.000.000	1.499.998	3.000.000	1,358
Shell Sort	100.000	3.089.145	3.000.012	5,981
Shell Sort	1.000.000	35.715.034	36.000.014	20,694

4.3 Vetor aleatório

Tabela 5: Desempenho médio dos algoritmos com vetor aleatório

Algoritmo	**n**	**Comparações**	**Movimentações**	**Tempo médio (ms)**
Bubble Sort	100.000	4.999.729.642	7.496.685.210	10.460,591
Bubble Sort	1.000.000	499.999.039.331	750.060.780.607	1.106.917,917
Insertion Sort	100.000	4.997.890.126	200.000	1.693,432
Insertion Sort	1.000.000	500.041.520.390	2.000.000	188.262,704
Selection Sort	100.000	4.999.950.000	299.969	3.689,811
Selection Sort	1.000.000	499.999.500.000	2.999.954	790.403,393
Merge Sort	100.000	0	1.668.928	0,638
Merge Sort	1.000.000	0	19.951.424	6,908
Quick Sort	100.000	2.262.511	1.160.771	9,203
Quick Sort	1.000.000	27.032.913	13.956.217	84,966
Heap Sort	100.000	180.676	392.033	0,351
Heap Sort	1.000.000	1.806.979	3.920.943	3,310
Shell Sort	100.000	7.142.823	3.000.012	9,553
Shell Sort	1.000.000	116.443.359	36.000.014	126,733

As tabelas comparam os algoritmos em relação à quantidade média de comparações, movimentações e tempo médio de execução. Os resultados foram separados por tipo de entrada, permitindo analisar o comportamento dos algoritmos em vetores ordenados, invertidos e aleatórios.

4.4 Gráficos de tempo de execução

Além das tabelas, foram gerados gráficos comparando o tempo médio de execução dos algoritmos. Como há grande diferença entre os tempos dos algoritmos quadráticos e dos algoritmos mais eficientes, foi utilizada escala logarítmica no eixo vertical. Essa escolha permite visualizar melhor os resultados sem que os algoritmos mais rápidos desapareçam visualmente diante dos tempos muito maiores dos métodos quadráticos.

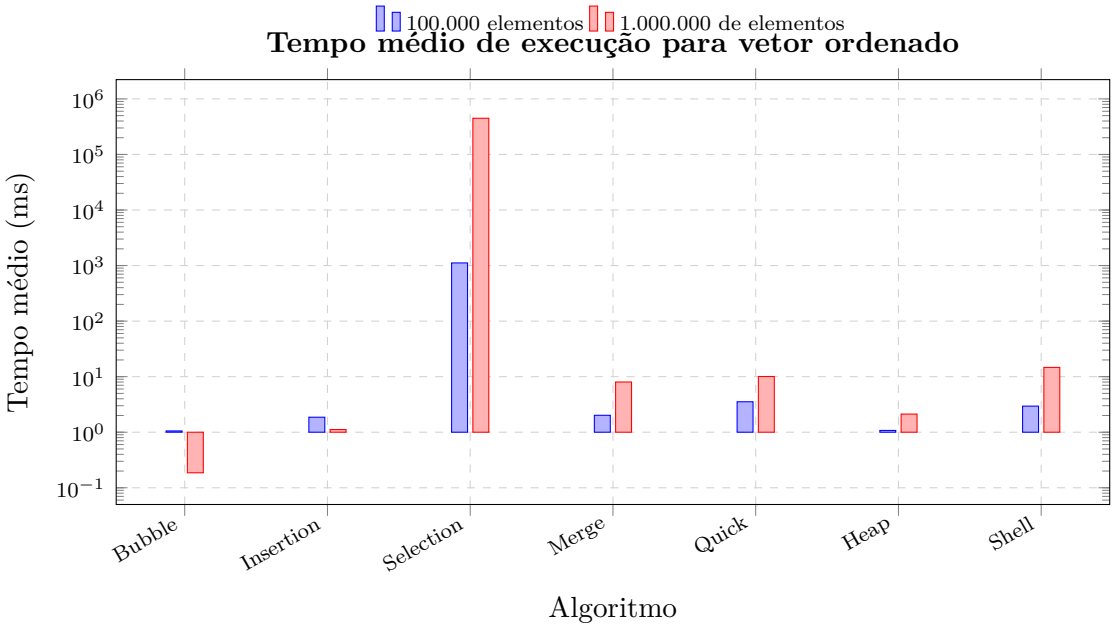


Figura 1: Tempo médio de execução dos algoritmos para vetor ordenado. A escala do eixo vertical é logarítmica para facilitar a visualização das diferenças entre os algoritmos.

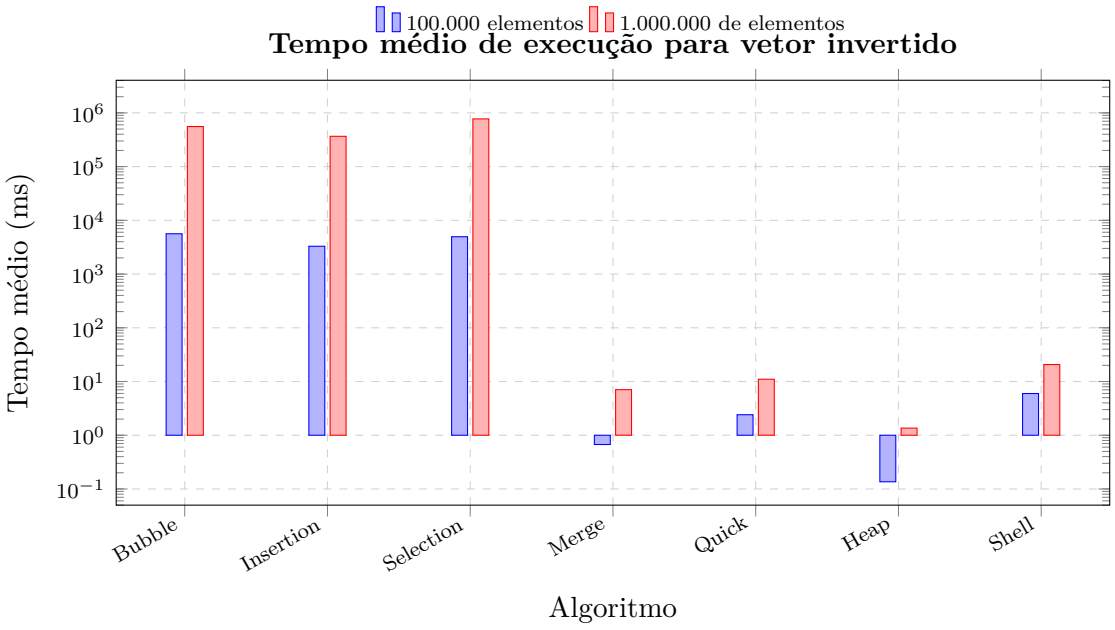


Figura 2: Tempo médio de execução dos algoritmos para vetor invertido. A escala do eixo vertical é logarítmica para facilitar a visualização das diferenças entre os algoritmos.

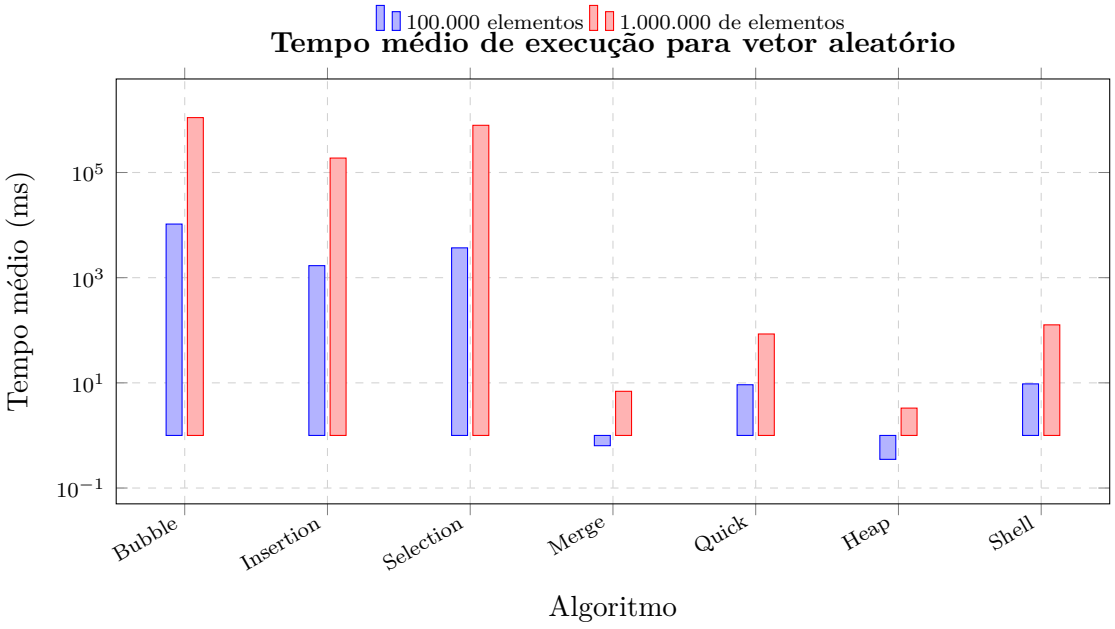


Figura 3: Tempo médio de execução dos algoritmos para vetor aleatório. A escala do eixo vertical é logarítmica para facilitar a visualização das diferenças entre os algoritmos.

5 Análise dos resultados

A análise dos resultados mostra que os algoritmos quadráticos, como Bubble Sort, Insertion Sort e Selection Sort, apresentam forte crescimento no número de operações quando o tamanho da entrada aumenta. Isso confirma a limitação prática de algoritmos

com complexidade $O(n^2)$ para grandes volumes de dados.

> **Destaque da análise:** os testes tornam evidente que a escolha do algoritmo depende diretamente do tamanho da entrada e da organização inicial dos dados. Métodos simples são úteis para fins didáticos, mas algoritmos como Merge Sort, Quick Sort, Heap Sort e Shell Sort tornam-se mais interessantes em problemas maiores.

No vetor já ordenado, Bubble Sort e Insertion Sort apresentaram comportamento melhor em relação aos demais cenários. Isso ocorre porque ambos conseguem se beneficiar de uma entrada que já está organizada. O Bubble Sort pode encerrar sua execução quando nenhuma troca é realizada, enquanto o Insertion Sort faz poucas movimentações.

No vetor invertido, os algoritmos quadráticos foram os mais afetados. Bubble Sort e Insertion Sort precisam realizar muitas trocas ou deslocamentos, pois praticamente todos os elementos estão distantes de suas posições finais. Esse cenário evidencia o pior caso desses métodos.

No vetor aleatório, os algoritmos mais eficientes se destacaram. Merge Sort, Quick Sort, Heap Sort e Shell Sort apresentaram comportamento mais adequado para grandes volumes de dados, principalmente quando comparados ao crescimento de tempo dos métodos quadráticos.

O estudo dos algoritmos adicionais foi uma parte relevante do trabalho, pois permitiu observar abordagens diferentes das apresentadas inicialmente em aula. O Heap Sort se mostrou interessante por utilizar a estrutura de heap, conectando o tema de ordenação a estruturas de dados mais avançadas, como filas de prioridade. O Shell Sort também se destacou por mostrar como uma alteração relativamente simples no Insertion Sort pode melhorar o desempenho prático.

De modo geral, os resultados práticos confirmam a expectativa teórica: algoritmos $O(n^2)$ são menos adequados para grandes entradas, enquanto algoritmos próximos de $O(n \log n)$ tendem a ser mais eficientes e escaláveis.

6 Código-fonte

O código-fonte foi desenvolvido em Java e organizado em classes separadas para cada algoritmo de ordenação. Também foram criadas classes auxiliares para armazenar métricas e executar os testes.

A estrutura principal do projeto foi organizada da seguinte forma:

```
src/work2/
Main.java
ExecutorTestesOrdenacao.java
```

`Sorter.java`
`SortMetrics.java`
`BubbleSort.java`
`InsertionSort.java`
`SelectionSort.java`
`MergeSort.java`
`QuickSort.java`
`HeapSort.java`
`ShellSort.java`

O código completo está disponível no GitHub:

https://github.com/GarrardFontanive/Estrutura-de-Dados/tree/main/src

A execução do programa gera dois arquivos principais:

- `resultados.csv`: contém os resultados individuais de cada execução;
- `medias.csv`: contém as médias de comparações, movimentações e tempo.

6.1 Trechos relevantes do código

Além da organização em classes separadas, o código foi comentado para indicar os pontos principais exigidos no trabalho: comparações entre elementos, trocas ou movimentações, divisão do problema nos algoritmos recursivos e combinação dos resultados.

Comparações entre elementos

As comparações são registradas sempre que dois valores do vetor, ou um valor e uma chave/pivô, são avaliados para decidir a ordem dos elementos.

Listing 1: Exemplo de comparação no Bubble Sort

```
metricas.incrementarComparacoes();

if (vetor[j] > vetor[j + 1]) {
    // Troca realizada quando os elementos estao fora de ordem
}
```

Trocas ou movimentações

As movimentações são registradas quando ocorre troca entre posições do vetor ou deslocamento de elementos.

Listing 2: Exemplo de troca entre elementos

```
int temp = vetor[j];
vetor[j] = vetor[j + 1];
vetor[j + 1] = temp;

metricas.adicionarMovimentacoes(3);
```

Listing 3: Exemplo de movimentação no Insertion Sort

```
vetor[j + 1] = vetor[j];
metricas.incrementarMovimentacoes();
```

Divisão do problema nos algoritmos recursivos

No Merge Sort, a divisão ocorre separando o vetor em duas metades, que são ordenadas recursivamente.

Listing 4: Divisão recursiva no Merge Sort

```
int meio = inicio + (fim - inicio) / 2;

mergeSort(vetor, auxiliar, inicio, meio, metricas);
mergeSort(vetor, auxiliar, meio + 1, fim, metricas);
```

No Quick Sort, a divisão ocorre por meio do particionamento em torno de um pivô.

Listing 5: Divisão por particionamento no Quick Sort

```
int pivo = vetor[inicio + (fim - inicio) / 2];

while (i <= j) {
    while (vetor[i] < pivo) {
        metricas.incrementarComparacoes();
        i++;
    }

    while (vetor[j] > pivo) {
        metricas.incrementarComparacoes();
        j--;
    }

    if (i <= j) {
        trocar(vetor, i, j, metricas);
        i++;
        j--;
```

```
18      }
19  }
```

Combinação dos resultados

No Merge Sort, depois da divisão recursiva, as duas metades ordenadas são combinadas por meio da intercalação.

Listing 6: Combinação das metades no Merge Sort

```
1  while (i <= meio && j <= fim) {
2      metricas.incrementarComparacoes();
3
4      if (auxiliar[i] <= auxiliar[j]) {
5          vetor[k] = auxiliar[i];
6          i++;
7      } else {
8          vetor[k] = auxiliar[j];
9          j++;
10     }
11
12     metricas.incrementarMovimentacoes();
13     k++;
14 }
```

Esses trechos resumem os pontos centrais da implementação e demonstram como o código permite a reprodução dos testes e a coleta das métricas utilizadas no relatório.

7 Conclusão

A realização deste trabalho permitiu comparar, na prática, diferentes algoritmos de ordenação e compreender como o desempenho varia conforme o tamanho da entrada e a organização inicial dos dados. A partir dos testes realizados, foi possível observar que algoritmos com lógica mais simples nem sempre são adequados para grandes volumes de informação.

Os algoritmos Bubble Sort, Insertion Sort e Selection Sort são importantes para fins didáticos, pois facilitam a compreensão dos conceitos iniciais de comparação, troca e movimentação de elementos. Entretanto, os resultados demonstram que esses métodos apresentam limitações significativas quando aplicados a vetores maiores, principalmente nos cenários de vetor invertido e vetor aleatório.

O Merge Sort e o Quick Sort apresentaram melhor desempenho geral em comparação aos métodos quadráticos. O Merge Sort se destaca pela regularidade de sua complexi-

dade $O(n \log n)$, enquanto o Quick Sort apresenta bom desempenho prático quando o particionamento do vetor ocorre de forma equilibrada.

Entre os algoritmos adicionais, o Heap Sort mostrou-se relevante por utilizar a estrutura de heap, conectando a ordenação ao estudo de estruturas de dados mais avançadas, como filas de prioridade. O Shell Sort também se mostrou interessante por representar uma melhoria prática do Insertion Sort, demonstrando como uma alteração na estratégia de comparação pode reduzir movimentações e melhorar o desempenho.

O estudo dos métodos de ordenação também contribui para outras áreas da computação, como bancos de dados, sistemas de busca, análise de grandes conjuntos de dados, processamento de arquivos, construção de relatórios, organização de listas e otimização de aplicações. Em todos esses casos, escolher um algoritmo adequado pode reduzir o tempo de processamento e melhorar a eficiência geral do sistema.

Conclui-se que a escolha do algoritmo de ordenação deve considerar o tamanho da entrada, o tipo de dado, a organização inicial do vetor, o tempo disponível para processamento, o número de operações realizadas e o uso de memória. Para vetores pequenos, algoritmos simples podem ser suficientes. Para grandes volumes de dados, algoritmos como Merge Sort, Quick Sort e Heap Sort são mais indicados. Já o Shell Sort pode ser considerado uma alternativa intermediária, com implementação relativamente simples e desempenho prático superior aos métodos quadráticos tradicionais.

Referências

[1] CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Introduction to Algorithms*. 4. ed. Cambridge: MIT Press, 2022.

[2] SHELL, Donald L. A high-speed sorting procedure. *Communications of the ACM*, v. 2, n. 7, p. 30–32, 1959.